[11] SPIDER: Tracking Gradients with Less Noise

C. Fang, C. J. Li, Z. Lin and T. Zhang — NeurIPS (2018)

SPIDER: Near-Optimal Non-Convex Optimization via Stochastic Path-Integrated Differential Estimator — RQ2, RQ3

Problem: Full gradients are expensive on large data, but individual stochastic gradients are noisy. **Contribution:** SPIDER builds a recursive estimator that tracks the full gradient using differences evaluated on *the same minibatch*. **Results:** Near-optimal first-order stationarity complexity; an extended SPIDER-SFO$^+$ method also targets approximate second-order stationarity.

1. Start with the finite-sum training problem

For n samples, write

$$f(x) = \frac{1}{n}\sum_{i=1}^{n} f_i(x), \qquad \nabla f(x) = \frac{1}{n}\sum_{i=1}^{n} \nabla f_i(x).$$

GD computes every term at every iteration; SGD uses $g_t = \nabla f_{i_t}(x_t)$ as a cheap but noisy estimate. Noise can dominate when the true gradient is small, so merely taking more SGD steps may not be efficient.

2. The estimator: subtract two correlated gradients

At periodic *refresh* steps SPIDER obtains a relatively accurate gradient estimate (full gradient in the finite-sum setting, or a sufficiently large batch in the online setting). Between refreshes it updates

$$\boxed{v_t = v_{t-1} + \frac{1}{|S_t|}\sum_{i\in S_t} [\nabla f_i(x_t) - \nabla f_i(x_{t-1})]}.$$

The crucial detail: use *the same sampled indices* S_t at both x_t and x_{t-1}. If nearby points have similar gradients, the difference has low variance even when either gradient alone is noisy.

3. Why cancellation helps — a concrete toy calculation

Suppose a sampled gradient has a shared random component, $g_i(x) = h(x) + \xi_i$. A fresh gradient at each point has noise. But with the same i,

$$g_i(x_t) - g_i(x_{t-1}) = h(x_t) - h(x_{t-1}) + (\xi_i - \xi_i),$$

and the shared noise cancels exactly in this teaching model. In real learning, it does not cancel perfectly, but smoothness controls the variance of the gradient *difference* as steps remain small.

4. How tracking enters optimization

SPIDER-SFO combines the estimator v_t with a normalized descent rule, schematically

$$x_{t+1} = x_t - \eta_t \frac{v_t}{\|v_t\|} \quad \text{when } v_t \neq 0.$$

The normalization helps control the distance between successive iterates, making the estimator's accumulated error manageable. The actual algorithm specifies adaptive step lengths, refresh schedules and termination conditions.

Core rates — read the tolerance carefully. For a gradient criterion $\mathbb{E}\|\nabla f(\hat{x})\| \lesssim \epsilon$ (or corresponding prescribed stationarity output), the paper reports stochastic-gradient evaluation complexity of order

$$\boxed{O\big(\min\{\sqrt{n}\,\epsilon^{-2},\, \epsilon^{-3}\}\big)}$$

for its finite-sum/online regimes, under their respective smoothness and stochastic-oracle assumptions. With SPIDER-SFO$^+$ and extra saddle-escape machinery, the second-order target $(\epsilon, O(\sqrt{\epsilon}))$ has complexity roughly

$$\widetilde{O}\Big(\min\{\sqrt{n}\,\epsilon^{-2} + \epsilon^{-5/2},\, \epsilon^{-3}\}\Big).$$

The first displayed rate is *FOSP*, not automatically a guarantee of saddle escape.

5. Assumptions, limitations and survey connection

The improvement relies on suitable smoothness, randomized sampling, controlled estimator error and refresh batches. Gradient evaluations, not optimizer state memory or GPU runtime, are counted. Second-order guarantees require the extended variant and additional conditions; they are not an automatic consequence of low-variance gradients. **Our takeaway.** SPIDER is important because it reduces the cost of knowing which direction is downhill; when joined with escape machinery it also helps reach a curvature-aware target. Variance reduction and saddle escape are connected but different goals. **Source:** https://papers.nips.cc/paper_files/paper/2018/hash/1543843a4723ed2ab08e18053ae6dc5b-Abstract.html.